Reconstructing Missing Indoor CO_2 Readings from a Building Sensor Log

MMA3001 Numerical Methods and Machine Learning – Individual Project

Nishad Adrekar
36181056

1 Engineering problem

1.1 Context

CO_2 shows whether a room is getting enough fresh air for the people in it. Too little fresh air makes people tired and less focused. Building managers and HVAC engineers use this data.

1.2 Problem

Sensors sometimes record nothing for a while, so readings go missing. If these gaps are filled badly, averages and peak values come out wrong, and ventilation decisions get made on wrong numbers.

1.3 Dataset

This project uses Monash Smart Infrastructure Dataset 2 and follows its listed direction, "reconstructing missing sensor measurements".

1.4 Research question

Which of four methods (forward-fill, linear interpolation, cubic spline and PCHIP) fills the gaps most accurately, and how does the accuracy change as the gaps get longer?

1.5 Intended use

An engineer analyzing stored logs, before the analysis and not live, would use the results to choose the best method and the longest gap that is safe to fill.

1.6 Scope

The project covers one sensor in one room, CO_2 only, and gaps from 10 minutes to 6 hours. It fills gaps after the fact; it does not predict readings live.

2 Inputs, outputs and data

2.1 Inputs

Table 1: Inputs

Input	Format	Units / domain	Handling of invalid input
Raw environmental data file (353 MB, provided by the teaching team)	A table with one row per stored record. Each row stores the reading, its unit and the time it was measured.	Readings with their measurement time (UTC) and unit	Missing file or missing column: the program stops with an error message
Sensor ID	Text, exact match	6012002000869	No matching rows: the program stops with an error message
CO_2 value	Number stored with each reading	ppm; accepted range 250–5000 ppm	Rejected and counted, with the reason (for example an

Input	Format	Units / domain	Handling of invalid input
			unreadable record, wrong unit, missing or non-numeric value, out of range)

Each reading's own measurement time is used. The raw file's name ("MayToDec2024") does not match the measurement dates (29 Feb – 13 Jun 2024), so the measurement times were used, not the file name. The time at which the server stored the row is not used. It mixes two UTC offsets because of daylight saving. It also keeps advancing while the sensor is silent.

Unsupported input: the interpolation methods do not extrapolate. If asked for a time outside the known readings, the program stops with an error message.

2.2 Outputs

All outputs below are results tables and figures in the repository.

Table 2: Outputs

Output	Meaning and intended use
Cleaned CO_2 series	The genuine readings, with their times. It is the input to all experiments and stays on the local machine.
Test results table	On the held-out test set, for each gap duration and method: number of gaps, error measures, share of gaps won and runtime.
Tuning results table	Results of hyperparameter tuning on the validation set (spline only).
Run records and figures	Run settings (periods, seed, software versions), a record of where computing time is spent, and the figures.
Function-level inputs/outputs	Each method returns one estimate per requested time, in the same units as the input. See the HTML code documentation.

2.3 Data audit findings

The environmental file has 179,447 rows from 5 sensors. Only two sensors contain live data. The other three (…227, …241 and …777) repeat one identical reading in every row. Sensor …326 also has live data, but it is not on the location sheet and covers a different period, so it was not used. This project uses sensor 6012002000869:

Source rows: 29,341. Genuine CO_2 readings: **13,800**. The other rows are stale copies of an earlier reading. Coverage: 90.8 % of the readings expected every 10 minutes. Native gaps: the sensor itself misses about 1 reading in 10. There are 1,390 such gaps.

The occupancy data were not used, because their rooms could not be linked to the CO_2 sensor.

3 Computational approach

3.1 Methods

Each method estimates the ground truth (the hidden readings) from the known readings around the gap. Interpolation has no training phase. Like KNN, each estimate uses only the nearest known readings. The known readings used on each side are called the **context (a hyperparameter)**. For example, a context of 2 means two known readings before the gap and two after.

Forward-fill (baseline model). Repeats the last reading before the gap. It uses only earlier readings, so it could run in real time. Its error grows as the true signal drifts away from that last value.

Linear interpolation. Draws a straight line between the two readings either side of the gap, using the NumPy library. The line always stays between those two values. So it can never overshoot, which means going above or below both nearby readings.

Cubic spline. A spline is a smooth curve made of short cubic pieces joined end to end. It passes through every known reading. It is computed with the SciPy library, using its standard "not-a-knot" end condition. On smooth signals it becomes much more accurate than a straight line as readings get closer. It has no limit on its shape, so it can overshoot.

PCHIP. PCHIP is a smooth curve that cannot overshoot. It sets the slope to zero at peaks and dips, so it never goes beyond the nearby readings.

3.1.1 Why these methods

They are the standard first tools for filling missing values in a time series. They are easy to interpret and cost almost nothing to run. The comparison tests one question: does the spline's extra accuracy on smooth curves carry over to a real, noisy sensor signal? PCHIP was added to test whether removing overshoots fixes the spline.

3.1.2 Assumptions

1. Each reading's own timestamp is correct.
2. The readings either side of a gap are genuine.
3. Readings are close enough together that the signal between them is fairly smooth. The spline relies on this the most.
4. Gaps placed at random behave like real dropouts.

3.2 Experimental design

Chronological split. The first 70 % of readings form the validation set: 9,660 readings, 29 Feb – 13 May 2024. The last 30 % form the held-out test set: 4,140 readings, 14 May – 13 Jun 2024. No reading is in both. Gaps and their context never cross the boundary.

Gap duration. A gap's duration D is a time window that starts at a genuine reading. D is 10, 30, 60, 180 or 360 min. Every reading in that window is hidden. For example, a 10-minute gap always hides exactly one reading. Its known neighbors were usually 20 min apart (173 of 200 gaps in the held-out test set). In the other 27, a native gap sat next to it, so they were 30 min apart.

Placement. Up to 200 gaps per duration are placed at random. A fixed random seed makes the run repeat exactly. Gaps never overlap. Long gaps did not always reach 200: the held-out test set has 187 three-hour and 92 six-hour gaps.

Fairness and no data leakage. All methods are scored on the same gaps, with the same context. The ground truth is removed before a method sees the data. A check in the program stops the run if any ground-truth value ever reaches a method. The automated tests confirm that this check works.

Hyperparameter tuning on the validation set. The only hyperparameter is the spline's context: 2, 4, 8, 16 or 32 readings each side. It was chosen on the validation set only, by the lowest average spline MAE. PCHIP needs no hyperparameter tuning. Its estimate depends only on the two readings on each side, so contexts 2 and 8 give identical predictions (this is tested).

3.2.1 Metrics

- MAE (mean absolute error): the average size of the error, in ppm, over all ground-truth readings.
- RMSE (root-mean-square error): like MAE, but large errors count more.

- Each comes with a 95 % interval, found by resampling whole gaps 1,000 times (a bootstrap).
- Maximum overshoot: how far an estimate went beyond the range of the context readings.
- % of gaps won: the share of gaps in which one method had a lower error than another.

3.3 Implementation

The code is written in Python, using the NumPy and SciPy libraries. It is on GitHub: https://github.com/nishad27-code/MMA3001-project. It has 113 automated tests. The 353 MB raw file is read once, in about 5 s, and a full experiment run takes about 40 s on a laptop. See the README and HTML documentation for how to run the code and for function details.

4 Verification

Exact answers. Linear interpolation reproduces a straight line to within 10^{-9}. The spline reproduces a cubic polynomial to within 10^{-8}, even with realistic time values.

Convergence order. The order says how fast the error shrinks as readings get closer together. The measured order is 2 ± 0.2 for linear interpolation. For the spline it is above 3.5 (theory: 4).

No data leakage. A ground-truth value deliberately set to 10^6 never reaches a method. The leakage check also stops a deliberately broken gap.

PCHIP stays within the boundary readings. On 300 noisy synthetic gaps, PCHIP never went beyond the two readings either side. The spline did overshoot on the same data.

Test suite. All 113 automated tests pass in the latest run.

5 Validation and results

5.1 What was validated

How well each method recovers genuine CO_2 readings that were hidden on purpose. The true values are known. So, this is a check against ground truth, on the held-out test set. That set was not used for any choice.

5.2 Why it is appropriate

It tests exactly the intended use: filling holes in a real log. It uses this sensor's real noise and behavior. No ground-truth value is ever used as input.

5.3 Did validation influence model selection?

Yes, in one way. PCHIP was added after the test results showed the spline overshooting. PCHIP has no hyperparameters, and the final choice (linear) did not change.

5.4 Hyperparameter tuning on the validation set

Context 2 had the lowest average spline MAE, so it was selected. It is the smallest context possible. With 4 points, the spline is a single cubic through them. More context made the spline worse (21.8 → 28.9 ppm).

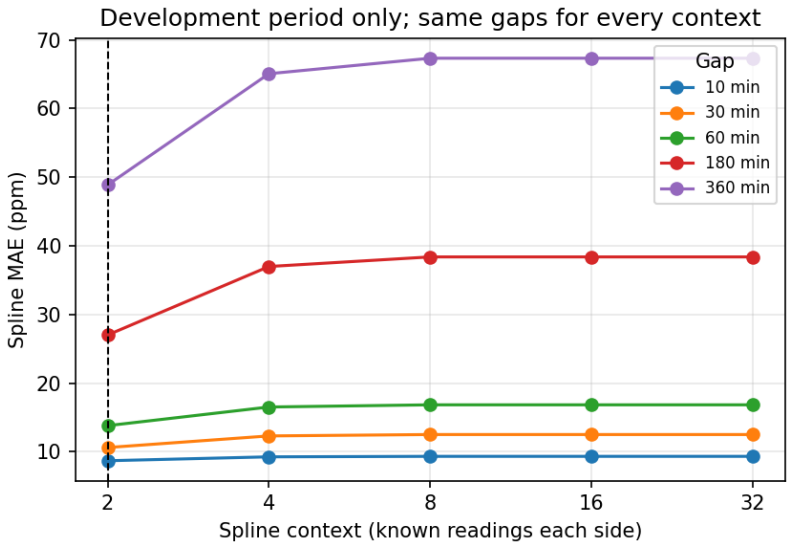


Figure 1: Spline MAE for each context size, validation set only. Every context uses the same 200 gaps per duration. The dashed line marks the chosen context.

Table 3: CO_2 results on the held-out test set (from the results tables in the repository; spline context 2). MAE is in ppm. The 95 % bootstrap interval over gaps is in brackets. "Boundary interval" is the average time between the known readings either side of a gap. The baseline model, linear and PCHIP never overshot (0.0 ppm at every duration).

Gap D	Gaps	Boundary interval (min)	Baseline (forward-fill) MAE	Linear MAE	Spline MAE	PCHIP MAE	Max overshoot, spline (ppm)
10 min	200	21.4	9.21 [8.31, 10.17]	**7.82** [7.08, 8.62]	8.97 [8.12, 9.88]	8.08 [7.32, 8.93]	6.5
30 min	200	41.3	9.78 [9.05, 10.53]	**7.98** [7.46, 8.52]	10.83 [10.05, 11.64]	8.38 [7.82, 8.96]	17.1
1 h	200	71.1	10.95 [10.12, 11.78]	**8.23** [7.73, 8.72]	13.01 [12.09, 13.99]	8.74 [8.19, 9.27]	27.1
3 h	187	191.8	15.05 [13.43, 16.80]	**10.15** [9.51, 10.84]	26.13 [23.92, 28.36]	11.11 [10.35, 11.94]	81.9
6 h	92	370.9	17.78 [15.07, 20.58]	**12.71** [11.20, 14.49]	47.76 [42.28, 54.12]	13.34 [11.61, 15.27]	179.9

Gap by gap, linear beat the baseline model in 57–70 % of gaps. The spline beat linear in only 5–31 % of gaps, and PCHIP in 22–27 %.

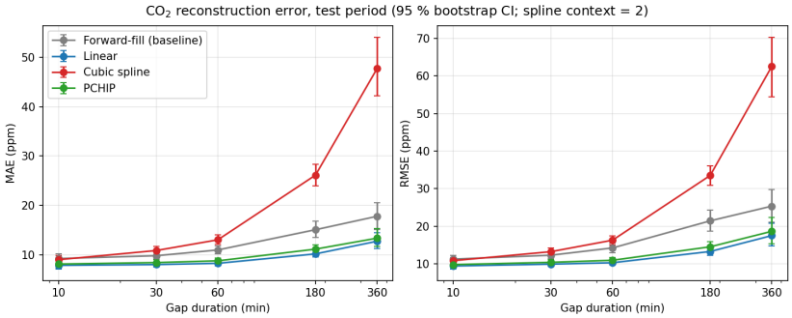


Figure 2: MAE and RMSE on the held-out test set against gap duration (log scale), with 95 % bootstrap intervals.

5.5 Signal context (held-out test set)

CO_2 averaged 470 ppm (SD 39 ppm). Most readings (5th–95th percentile) were between 428 and 555 ppm, and the maximum was 634 ppm. Consecutive readings, 10 minutes apart, differed by 9.0 ppm on average. Readings are reported until 1 ppm. This sets a floor on the error. The 10-minute linear MAE (7.8 ppm) is already below that typical change. So a single missing reading cannot be estimated much better by its neighbors.

5.6 Acceptance criterion

A method passes if its MAE is below the typical 9.0 ppm change between consecutive readings. Linear and PCHIP pass for gaps up to 1 h. The spline passes only at 10 min (8.97 ppm), and only just. The baseline model never passes.

5.7 Unsupported inputs and failure modes

The methods refuse to extrapolate beyond the known readings. Gaps longer than 6 h were not tested. The spline can overshoot badly, by up to 179.9 ppm on 6-hour gaps.

5.8 What the validation cannot establish

Performance in rooms with high or fast-changing CO_2. This room never went above 634 ppm. Performance in other seasons, sensors or buildings. Performance on real dropouts, if they do not happen at random times. For example, outages caused by power or network failures could behave differently. Whether the measurement timestamps themselves are correct.

6 Comparison and performance

6.1 Accuracy

Linear interpolation is the most accurate method at every duration tested. The spline is worse than the baseline model (forward-fill) from 30 minutes upward.

6.2 Trade-offs

Table 4: Trade-offs

Method	MAE at 1 h (ppm)	MAE at 6 h (ppm)	× change, 10 min → 6 h	Max overshoot (ppm)	Runtime ratio vs linear	Real-time capable?	Complexity
Baseline (forward-fill)	10.95	17.78	×1.9	0.0	about 1×	Yes	Trivial
Linear	8.23	12.71	×1.6	0.0	1	No	Simple
Cubic spline	13.01	47.76	×5.3	179.9	about 7×	No	Moderate
PCHIP	8.74	13.34	×1.7	0.0	about 8.5×	No	Moderate

6.3 Runtime

Absolute times were not stable between benchmark runs on this laptop, so only ratios to linear are reported. Forward-fill takes about as long as linear (about 1×). The spline takes about 7× as long, and PCHIP about 8.5×. Absolute times are tens to hundreds of microseconds per gap and depend on the machine.

6.4 Profiling and FLOP estimate

Profile. For the spline, most of the time is spent setting up the spline, not calculating. Our own input checks are the next biggest cost for all methods.

Arithmetic per gap. With context 2 there are 4 known points and at most 36 ground-truth readings to estimate. Linear: about 5 floating-point operations (FLOPs) per estimated reading, so under 200 per gap. Spline: about 40 FLOPs to set up the curve (one small system of equations), plus about 8 per estimated reading. That is under 400 per gap.

Implication. At tens to hundreds of microseconds per gap, the spline does at most a few million FLOPs per second. A laptop core can do about 10^{10} per second. So the time goes on Python and library overhead, not on arithmetic.

Roofline view. The methods sit far below both hardware limits, the arithmetic limit and the memory-bandwidth limit, so Python overhead sets the speed.

Memory. Memory use is negligible: each gap uses only 4 known readings.

Engineering implication. Filling all 1,390 native gaps in the log takes seconds at most with any method. So computing costs do not separate the methods. The choice should rest on accuracy and robustness.

6.5 Final design decision

To fill gaps in this sensor's CO_2 log after the fact, use **linear interpolation**. Fill gaps up to 1 h, the longest duration that passed the acceptance criterion. Flag longer gaps as unreliable. Mark all filled values as reconstructed. Do not use an unconstrained cubic spline on noisy 10-minute CO_2 data.

7 Limitations and lessons learned

7.1 Why the spline fails here

The spline's $O(h^4)$ advantage assumes the signal is smooth at the sampling scale. At 10-minute sampling, this CO_2 signal varies by about 9 ppm between consecutive readings, either measurement noise or genuinely fast fluctuation, while longer-term trends are small. In ML terms, the spline is overfitting the noise. It treats the noisy boundary readings as if they define a true slope and curvature. It continues that curvature across the gap, so overshoot rises from 6.5 ppm at 10 min to 180 ppm at 6 h. Linear interpolation cannot leave the range of its two boundary values, which makes it robust. The temperature signal is smoother. There, PCHIP was slightly better than linear at 3–6 h, by about 0.02 °C. That is below the sensor's 0.2–0.3 °C reading steps and is consistent with this explanation.

7.2 Why PCHIP lost

PCHIP removed the overshoot, but it was still slightly less accurate than linear interpolation. Its slope at each end of the gap is estimated from the next reading, and those readings are noisy. So, its curve follows the noise instead of a real trend. A straight line uses no slopes, so it is less affected by noise.

7.3 Why long gaps are harder

All methods only see the edges of a gap. Real changes inside it, such as occupancy events, ventilation cycles or day–night patterns, are invisible to them. The boundary readings are also single noisy

samples, so any trend inferred from them is uncertain. As the gap gets longer, the true signal has more time to depart from what the boundaries suggest.

7.4 Data-quality lesson

Most rows in the raw file were stale copies, and three sensors had no live data at all. Without stale-reading detection, a naive analysis would have treated constant repeated values as real measurements and judged reconstruction on fake data.

7.4 Limitations

- One sensor in one low-occupancy room (labelled Compactus upper).
- 3.5 months of data, with a one-month held-out test set in late autumn 2024.
- Artificial gaps are placed at random.
- The device's accuracy specification is unknown.
- The room location comes from a workbook that was not independently verified.

7.5 Possible improvements (not implemented)

- A smoothing spline or local regression to suppress noise before interpolating.
- A daily-profile or seasonal component for long gaps.
- Multivariate reconstruction using humidity or temperature.
- Validating on further sensors if the teaching team can supply repaired data.

7.6 Personal lessons learned

I learned that raw sensor data cannot be trusted without checking it. More than half of the rows were stale copies, and three sensors had no live data. If I hadn't checked, the results would have been less reliable.

I was surprised that the simplest method, a straight line, was the most accurate. On noisy data, a more complex method is not always better.

Next time I would keep the methods and maths at the unit's level from the start, instead of simplifying it at the end.

8 AI-use reflection

I used Claude (Anthropic) through Claude Code, which worked directly in my project folder, and a separate Claude Chat session.

Claude Code wrote about 90% of the code and tests, ran the experiments and drafted most of the report. The chat session explained concepts where I reviewed Claude Code's changes and approved them and helped me simplify the report. Section 1, the personal lessons and this reflection are based upon by own words, with Claude helping with the wording.

I used AI because the project needed a large amount of code and tests, I had limited time and was less confident with coding tools such as packaging, testing and documentation.

Each change was shown to me as a plan before it was saved, and I approved it step by step after thoroughly examining if it had followed my roadmap.

A separated Claude session reviewed the code and results. The code is checked by 113 automated tests, and the report's numbers were checked against the results files.

The important decisions were mine: following Dataset's 2's "reconstructing missing sensor measurements" direction, the scope, adding PCHIP, simplifying the statistics to match the unit and keep raw data out of GitHub.

The AI also made mistakes. It made the project more complex than the unit required, for example with advanced bootstrap statistics, which I removed. Some of its claims were wrong and were caught, such as that overshoot grows with gap length cubed. It once saved a change before I approved it, and its first timing measurements were unreliable and had to be redone.

AI let me build a larger project than I could on my own. However, its advanced maths made the project confusing at first. I now understand why linear interpolation was most accurate and how the validation and held-out test set were used. Next time I would keep the work at the unit's level from the start.

References

Monash Smart Infrastructure, *MMA3001 Dataset – MSI Occupancy and Environmental* (provided by the MMA3001 teaching team, 2026).

Virtanen, P. et al. (2020). SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods* 17, 261–272.

Harris, C. R. et al. (2020). Array programming with NumPy. *Nature* 585, 357–362. de Boor, C. (2001). *A Practical Guide to Splines* (rev. ed.).

Springer. Burden, R. L., Faires, J. D. & Burden, A. M. (2016).

Numerical Analysis (10th ed.). Cengage Learning. Efron, B. & Tibshirani, R. J. (1993).

An Introduction to the Bootstrap. Chapman & Hall. Fritsch, F. N. & Carlson, R. E. (1980).

Monotone piecewise cubic interpolation. *SIAM Journal on Numerical Analysis* 17(2), 238–246.